MLOps in Data Platforms

In the theoretical part of the class, we discussed how Machine Learning should not be treated as an isolated activity outside the data platform.

A model is not just something trained once in a notebook. In a production-oriented data platform, ML is part of a larger pipeline:

Raw data ⟶ Clean data ⟶ ML features ⟶ Model training ⟶ Model evaluation ⟶ Predictions ⟶ Served data product

By the end of this exercise, you should be able to:

- implement Flyte tasks and workflows for ML-related data pipelines;
- connect Flyte tasks to Trino;
- read and write Iceberg tables through Trino;
- create a Silver table from Bronze data;
- create a Gold ML feature table from Silver data;
- train a simple scikit-learn model using data from the platform;
- log parameters, metrics, and model artifacts to MLflow;
- load a trained model from MLflow;
- generate batch predictions;
- write prediction outputs back into a Gold table.

Part 1 – Basic MLOps with the Titanic dataset

In this practical exercise, you will implement a simplified version of that lifecycle using the Titanic dataset.

The goal is not to build the best possible ML model: the goal is to understand how ML can be integrated into a data platform using the same principles we already discussed:

- layered data architecture;
- Bronze, Silver and Gold tables;
- workflow orchestration;
- reproducible execution;
- feature generation;
- model training;
- experiment tracking;

- batch prediction;
- prediction outputs as data products.

You will implement four Flyte workflows:

1. Bronze ⟶ Silver cleaning workflow
2. Silver ⟶ Gold ML feature generation workflow
3. Model training workflow
4. Batch prediction workflow

You are working with the Titanic dataset inside a lakehouse-style data platform. Your objective is to build a simplified ML pipeline that predicts whether passengers survived.

The pipeline should:

- clean the raw Titanic data and publish it to Silver;
- generate ML features in Gold;
- train a Random Forest model;
- log the model and metrics to MLflow;
- load the trained model;
- generate predictions;
- publish the predictions to a Gold prediction table.

At the end of the exercise, you should have four Python files:

- `titanic_clean_data.py`
- `titanic_generate_ML_features.py`
- `titanic_train_ML.py`
- `titanic_predict.py`

Each file should define one Flyte workflow:

- `titanic_cleaning_workflow` – reads raw Titanic data from Bronze, removes records with missing age, and writes the cleaned records into the Silver layer. The purpose of this workflow is to simulate a basic data cleaning step.
- `titanic_generate_ML` – transforms the cleaned Silver data into a Gold table suitable for ML training. This workflow creates a simplified feature table.
- `titanic_training_workflow` – Create a Flyte workflow that reads the ML feature table from Gold, trains a scikit-learn model, evaluates it, and logs the result to MLflow. This workflow represents the training phase of the ML lifecycle.
- `titanic_prediction_workflow` – loads the trained model from MLflow, applies it to Titanic records, and writes predictions into a Gold prediction table. This workflow represents a simplified batch scoring pipeline.

The workflows should be executable independently.

Part 2 - Distributed Cross-Validation with Flyte

In the previous practical exercise, you implemented a simplified ML pipeline with Flyte: data preparation, model training, model logging, and batch prediction. In this exercise, we move one step further and explore a more advanced orchestration pattern: distributed model evaluation.

Instead of training and evaluating a model once, you will implement a cross-validation workflow where each fold is evaluated independently. Each fold will run as a separate Flyte task, allowing the platform to execute multiple evaluations in parallel.

This exercise illustrates an important idea: Some ML workloads are naturally parallel.

Cross-validation is a good example because each fold can be trained and evaluated independently, as long as all folds are later summarized into a final result. The workflow you build will use the Heart dataset from Trino, train a Logistic Regression model, evaluate it across multiple folds, log results to MLflow, and finally train a final model on the full dataset. The expected workflow structure is based on the provided target implementation.

The goal is to build a distributed cross-validation workflow for a binary classification problem.

Part 3 - Distributed Grid Search with Flyte

In the previous exercise, you distributed cross-validation folds across Flyte tasks. In this exercise, you will distribute a different kind of ML workload: hyperparameter search.

Hyperparameter search is another naturally parallel ML pattern. Each candidate configuration can be trained and evaluated independently. Once all configurations have been evaluated, the workflow can compare the results, select the best configuration, and train a final model.

You will implement a distributed grid search workflow using the Iris dataset, a Decision Tree classifier, Flyte map_task, and MLflow. The goal is not only to find the best model. The goal is to understand how an orchestrator can coordinate parallel ML experiments and then reduce the results into a final production artifact.

You are free to use any parameter grid you want. Each candidate configuration should create one MLflow run. The run name should identify the configuration. Each run should log all evaluation metrics. Evaluate each model using cross-validation, and then pick the best model and train a final model on the full dataset.

Check the results on MLFlow.